



Computational Finance

Exam - April Session

| Matteo Scarbini 1036580

April 9, 2023

Contents

1	Securitization	3
1.1	Preliminary Tests	3
1.2	Default times	5
1.3	Default counter	5
1.4	Tranches' Payoffs	7
1.5	Monte Carlo Results	9
1.5.1	Performance	10
2	Path Dependent Options	11
2.1	Asian Options	11
2.1.1	Black-Scholes	11
2.1.2	Heston	13
2.1.3	Variance Gamma	15
2.1.4	Jump Merton	16
2.2	Barrier Options	16
2.2.1	Black-Scholes	17
2.2.2	Heston	18
2.2.3	Variance Gamma	18
2.2.4	Jump Merton	19

1 Securitization

1.1 Preliminary Tests

The bond that is priced in this exercise is generated from a securitization process and can be categorized as an asset backed security (ABS). Therefore, in order to obtain all the information to price this product, it is necessary to simulate the evolution of the underlying assets. For this purpose we apply the function `BSn.trj()` to generate trajectories for 20 assets, according to the Black and Scholes model. Before proceeding with the core of the exercise, we perform some preliminary tests to check that our code is working and the data is correct. First we check that the correlation matrix is symmetric and positive definite (which means that all of its eigenvalues are positive). Then we perform the martingale test to verify that the generated trajectories are martingale. In particular for each trajectory we display a plot with errorbars, so that the condition can be checked with a quick look at the graphs: the red line represents $S_i(0) = 1$, the green markers represent the estimate of $\mathbb{E}[S_i(t)]$ at each monthly time step ($0 \leq t \leq 2Y$), and the green vertical lines represent the confidence interval for the mean, given by $\pm 3 \cdot \frac{\sigma}{\sqrt{J}}$. The martingale condition is respected, as it can be seen in figure 1.

After the preliminary tests have given positive results we need to compute the interpolated discount curve that will be used to discount the bond's cash flows. In order to do that, we defined a function that loops over the time steps that we need (24 monthly steps in 2 years), and calls the `P_0t` method on the `dc` object.

```
1 def interpolated_P0t(dc, T, Nb):
2
3     times = np.linspace(0, T, Nb, endpoint = True)
4     P_0t = np.ndarray(shape = (len(times),), dtype = "f" )
5
6     k = 0
7     for t in times:
8         P_0t[k] = dc.P_0t(t)
9         k += 1
10
11     return P_0t
```

We also decided to check whether the resulting interpolated curve was coherent with the provided data, which means that the zero coupon bond prices should be strictly decreasing with respect to maturity.

```
1 def check_increasing_discount(P0t):
2     if np.all(np.diff(P0t) < 0):
3         print("The zero coupon bond prices are strictly decreasing with respect to maturity.")
4     else:
5         print("The discount curve is not valid.")
```

Martingale Test

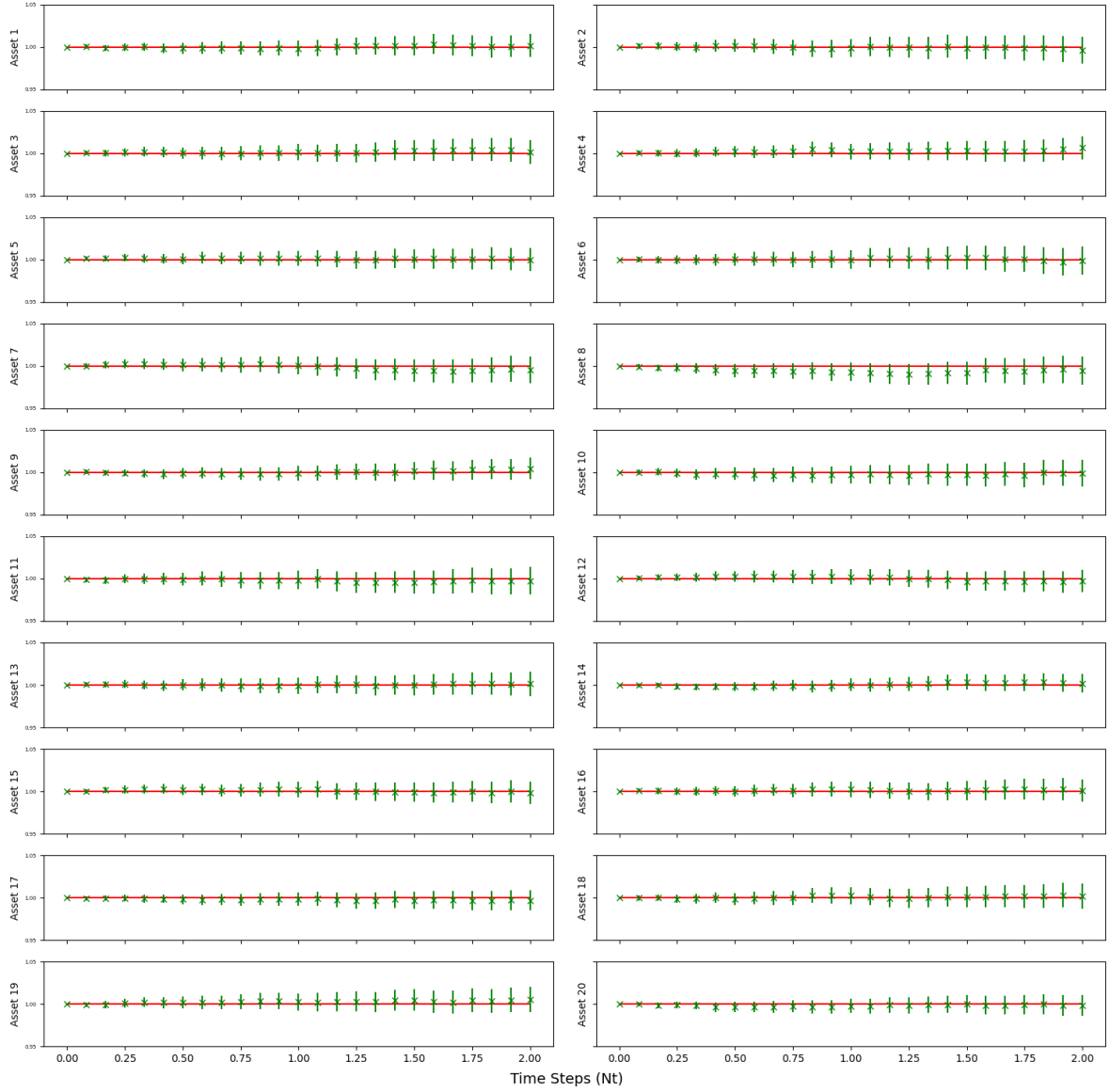


Figure 1: Martingale check for 20 assets.

1.2 Default times

In this exercise's framework, an asset is considered defaulted if it goes below 20% of its initial value, and that company will stop paying dividends. Therefore we need to verify if a company fails and when. Although the bond is characterized by monthly cash flows and it would be sufficient to check the default condition at the coupon dates, we decided to check whether the default happens more frequently (between the coupon dates). For this purpose we defined the function `t_default()`, which is shown in the box below.

The algorithm works as follows:

1. An array `mask` of the same dimension of `S` is created. This matrix has value "False" in the spots where the default condition is verified, while the rest of the values is set to "True" (line 14);
2. With `np.argmin()` we obtain a two dimensional array containing the time index of the first default for each asset and trajectory (line 17);
3. Through the if statement we translate the time indexes into monthly figures. In particular with `np.ceil()` we obtain the coupon date that comes immediately after the default event. Moreover, this part of the code is designed to work also if the frequency of the underlying is not monthly (lines 20-21).

```
1 def t_default(S, T, Nt, Nb, thres):
2
3     '''
4     Returns an array of shape (J,Q) containing the first month in
5     which an asset satisfies the default condition for each trajectory
6
7     @ S      multidimensional BS trajectories
8     @ T      maturity
9     @ Nt     number of time steps in the underlying trajectories
10    @ Nb     number of time steps for coupon payments
11    @ thres   percentage of initial value below which a default happens
12    '''
13
14    mask = np.where(S/S[0] < thres, False, True)
15
16    # Returns the time index of the first default for each asset and trajectory
17    t_default = np.argmin(mask, axis = 0)
18
19    # Translates the time indexes into monthly figures
20    if Nt % Nb != 0:
21        t_default = np.ceil( t_default / (Nt / Nb) )
22
23    return t_default
```

1.3 Default counter

The next step is to determine how many assets have failed by a certain time. In fact, in order to know which of the different tranches is affected, we need to compute the cumulative percentage of defaults for each time step.

Inside the for loop at line 13, the `counter` counts along the columns of `times_default` (that has shape

(J,Q)) how many assets default exactly at a specific time. Then, dividing by the total number of assets we obtain the percentage of defaults at that specific moment.

Subsequently `cum_counter` keeps track of the cumulative percentage of defaults for each time step and trajectory (the resulting array has shape (Nb, J)).

```

1 def cum_counter(Nb, Nt, J, S, T, thres):
2
3     '''
4     Returns an array of dimensions (Nb, J) containing the cumulative percentage
5     of defaulted assets at each time step
6     '''
7
8     times_default = t_default(S, T, Nt, Nb, thres)
9
10    cum_counter = np.zeros(shape = (Nb, J))
11    counter = np.zeros(shape = (J,))
12
13    for n in range(Nb):
14
15        # we rule out 0 valued times of default
16        counter += np.count_nonzero(times_default == n + 1, axis = 1) / np.shape(S)[2]
17        cum_counter[n] = counter
18
19    return cum_counter

```

In order to verify the correct functioning of the cumulative counter, we designed a small test with an array of 2 trajectories and 5 assets (importing Corrs5x5); Nb is set equal to 12 and the default threshold is set to 0.8. Below are displayed the results.

```

Default times (J,Q):
[[0. 4. 0. 0. 0.]
 [1. 5. 0. 3. 0.]]

```

```

Cumulative defaults (Nb,J):
[[0. 0.2]
 [0. 0.2]
 [0. 0.4]
 [0.2 0.4]
 [0.2 0.6]
 [0.2 0.6]
 [0.2 0.6]
 [0.2 0.6]
 [0.2 0.6]
 [0.2 0.6]
 [0.2 0.6]
 [0.2 0.6]]

```

Keeping in mind that the rows represent the time steps and the columns are the different trajectories, we can briefly explain the test output. Looking at the default times of the second trajectory, the first failure happens on the first month and, in fact, the value on the first row and second column of the cumulative counter is equal to 0.2 (1/5 of the assets has defaulted). The counter then maintains the same value until there is another default, which happens on the third month: on the third row of the second column it assumes value 0.4.

It can also be noticed that the first trajectory has only 1 non-zero value, which means that only one assets defaults in the whole period. As a consequence the maximum value of the cumulative counter is 0.2.

1.4 Tranches' Payoffs

Once the functions that give us all the information about the defaults have been defined, we are finally able to compute the payoffs of the bond's tranches. Each tranche is affected in a particular way by the cumulative percentage of defaults, and the difference is determined by the attachment and the detachment level. This is exactly what the function `tranches()` does in a vectorized way:

1. The effect of defaults on the reimbursement of the notional and the coupon payments are computed in an “unbounded” manner by the `loss` dictionary (*line 45*). For each different tranche i , we have:

$$loss_i = \frac{cum_defaults - attachment_i}{detachment_i - attachment_i}.$$

The marginal effect of one additional default is therefore inversely proportional to the size of the tranche (at the denominator): 1/4 for the junior and the mezzanine tranches, and 1/12 for the senior tranche. Moreover, the values we obtain from the above formula could easily be negative or greater than 1, while instead we should obtain numbers between 0 and 1 (the notional cannot increase or be negative).

2. This issue is solved by two masks at *lines 48-49*, that set all negative values of the `loss` dictionary equal to 0 and all the values exceeding the upper bound equal to 1.
3. The discounted cash flows at each time step are computed in a vectorized way, with the use of `np.einsum()`, and taking into account the loss represented by `mask2`. At the last time step the reimbursement is added to the coupon payment (*lines 52-53*).
4. Finally the cash flows are summed along the time axis to obtain the payoff of the tranches (*line 55*).

```

1 def tranches(R, att, det, Nb, Nt, T, cal_N, J, thres, So, sigma, C, dc, Seed):
2
3     '''
4     see the notebook for the description of the parameters
5     '''
6
7     rand = np.random.RandomState()
8     rand.seed(Seed)
9     M = BSn_trj(rand, Nt, So, C, sigma, T, J)
10
11     P0t = interpolated_P0t(dc, T, Nb + 1)[1:]
12
13     # From martingale M to S
14     D_tT = interpolated_P0t(dc, T, Nt + 1)
15     S = np.einsum('ijk,i -> ijk', M, 1/D_tT)
16
17     tau = T/Nb
18     cum_defaults = cum_counter(Nb, Nt, J, S, T, thres)
19
20     loss = np.ndarray(shape = (Nb, J))

```

```

21
22     loss  = {}
23     mask1 = {}
24     mask2 = {}
25     payoff = {}
26
27     for i in range(3):
28
29         # computes the loss that affects only the i-th tranche
30         loss[i] = (cum_defaults - att[i])/(det[i] - att[i])
31
32         # losses greater than 1 are set to 1, while negative losses are set to 0
33         mask1[i] = np.where(loss[i] > 0, loss[i], 0)
34         mask2[i] = np.where(mask1[i] < 1, mask1[i], 1)
35
36         # cash flows at each time step
37         payoff[i] = np.einsum('j,jk -> jk', P0t * tau * R[i] * cal_N , (1-mask2[i]))
38         payoff[i][-1] += P0t[-1] * cal_N * (1-mask2[i][-1])
39
40         payoff[i] = np.sum(payoff[i], axis = 0)
41
42     return payoff[0],payoff[1], payoff[2]

```

In order to be more clear it is useful to show how the loss is computed through a numeric example regarding the mezzanine tranche. In particular the cumulative defaults from the test defined above are used.

"Cumulative defaults"	"loss"	"mask1"	"mask2"
array([[0. , 0.2],	array([[-1. , 0.],	array([[0. , 0.],	array([[0. , 0.],
[0. , 0.2],	[-1. , 0.],	[0. , 0.],	[0. , 0.],
[0.2, 0.4],	[0. , 1.],	[0. , 1.],	[0. , 1.],
[0.2, 0.4],	[0. , 1.],	[0. , 1.],	[0. , 1.],
[0.2, 0.6],	[0. , 2.],	[0. , 2.],	[0. , 1.],
[0.2, 0.6],	[0. , 2.],	[0. , 2.],	[0. , 1.],
[0.2, 0.6],	[0. , 2.],	[0. , 2.],	[0. , 1.],
[0.2, 0.6],	[0. , 2.],	[0. , 2.],	[0. , 1.],
[0.2, 0.6],	[0. , 2.],	[0. , 2.],	[0. , 1.],
[0.2, 0.6],	[0. , 2.],	[0. , 2.],	[0. , 1.],
[0.2, 0.6],	[0. , 2.],	[0. , 2.],	[0. , 1.],
[0.2, 0.6]])	[0. , 2.]])	[0. , 2.]])	[0. , 1.]])

As we can see from the second array, the losses are negative when the cumulative percentage of defaults is below the attachment of the tranche, they are 0 when the percentage is exactly equal to the attachment, and they are greater than 1 when the percentage exceeds the detachment.

In reality, when the cumulative percentage of defaults is below the attachment level the losses should be 0, because the tranche is not affected. Moreover, when the percentage exceeds the detachment, the loss should be exactly 1, because the tranche should be completely exhausted. As mentioned before, this issue is solved by two masks that use `np.where()`: `mask1` sets the negative values in `loss` equal to 0, while `mask2` sets the values of `mask1` greater than the upper bound equal to 1 (see the last two arrays above).

1.5 Monte Carlo Results

The function `tranches()` returns the payoffs of the different tranches for each trajectory of the underlying assets. Therefore the last step is to compute the mean, the standard deviation, and the Monte Carlo error. The structure of our function `mc_securitization()` is similar to `do_mc()` of the CFLib, so the code is not displayed and the focus of this section is on the results.

```
@          junior  mezzan  senior
@ R          0.075   0.025   0.012
@ attachment 0.000   0.200   0.400
@ detachment 0.200   0.400   1.000

@ Nb          24.0000
@ Nt          24.0000
@ T           2.0000
@ Notional    100.0000
@ Default threshold 0.2000

@ elapsed time for 16384 iterations: 0.53928 sec.
@ elapsed time for 65536 iterations: 2.45652 sec.
@ elapsed time for 262144 iterations: 11.61655 sec.
@ elapsed time for 1048576 iterations: 248.35086 sec.

      J          Junior +/-      MC-err
16384      101.148933 +/-  9.9685e-02
65536      101.084892 +/-  5.0133e-02
262144      101.074336 +/-  2.5055e-02
1048576      101.128240 +/-  1.2486e-02
#----

      J      Mezzanine +/-      MC-err
16384      98.308289 +/-  0.0000e+00
65536      98.308289 +/-  0.0000e+00
262144      98.307752 +/-  2.1949e-04
1048576      98.307729 +/-  1.2084e-04
#----

      J          Senior +/-      MC-err
16384      95.793901 +/-  0.0000e+00
65536      95.793901 +/-  0.0000e+00
262144      95.793901 +/-  0.0000e+00
1048576      95.793901 +/-  0.0000e+00
#----
```

We immediately notice that the value of the tranches is increasing with their riskiness: Senior < Mezzanine < Junior. This happens because the default threshold is really low and it is unlikely for an asset (considering the provided volatilities) to go below 20% of its initial value. As a consequence, the junior tranche benefits from higher coupon payments and in most of the trajectories also the notional is repaid at maturity. Moreover, observing the behavior of the error of this tranche, which is cut approximately in half when the number of simulations is increased by four times, we can affirm that our Monte Carlo simulation is working properly.

With regard to the less risky tranches, they are almost not affected by the defaults at all. This can be deducted by looking at their Monte Carlo errors, which are 0 for many different values of J. In particular, the cumulative percentage of defaults never reaches the attachment of the senior tranche, and its payoff does not vary across the different trajectories. For the mezzanine tranche, instead, the MC-error is 0 for the two lowest numbers of simulations, while it is different from 0 when the number of simulations is higher. An explanation to this comes from the fact that when more trajectories are generated, the occurrence of defaults becomes more likely.

In order to show the effects of an higher number of defaults on the three tranches, we implemented the possibility of changing the default threshold. The results produced by a threshold of 0.6 are displayed below.

J	Junior	+/-	MC-err
16384	8.558626	+/-	9.0990e-02
65536	8.554023	+/-	4.5552e-02
262144	8.568790	+/-	2.2755e-02
1048576	8.597800	+/-	1.1411e-02
#----			
J	Mezzanine	+/-	MC-err
16384	49.925731	+/-	2.7682e-01
65536	50.087255	+/-	1.3896e-01
262144	49.992754	+/-	6.9459e-02
1048576	50.032971	+/-	3.4741e-02
#----			
J	Senior	+/-	MC-err
16384	94.249229	+/-	3.6988e-02
65536	94.236215	+/-	1.8485e-02
262144	94.227656	+/-	9.2719e-03
1048576	94.228576	+/-	4.6351e-03
#----			

Contrary to what happened before, the value of the tranches is now decreasing with the level of riskiness: Junior < Mezzanine < Senior. With an higher threshold, more defaults are observed and all of the tranches have a higher probability of being affected.

The junior tranche is the most affected and the price is well below par because, since many assets default before maturity, the notional is almost never reimbursed. Also the mezzanine tranche has a lower value than the previous experiment (with 0.2 threshold). As expected, due to its high attachment, the less affected tranche is the senior, whose price is similar as before. Finally, we notice that this time all the Monte Carlo errors are different from zero and are cut in half when the number of simulations is multiplied by four times.

1.5.1 Performance

The code is quite slow, in fact it takes more than four minutes to perform one million simulations. Therefore we implemented a small test to identify which part of the code is the most computationally expensive. As we were expecting, the generation of the multidimensional Black-Scholes trajectories alone accounts for the majority of the elapsed time: 165 seconds.

2 Path Dependent Options

In the following sections two different families of options will be analyzed, asian options and barrier options. In both cases we will take into account trajectories for the underlying asset generated according to four different models: Black-Scholes, Heston, Variance Gamma and Jump Merton.

Before proceeding with the options pricing, we perform the usual martingale test on the trajectories generators, similarly to 1. The plots with errorbars for the four models show that the condition is respected.

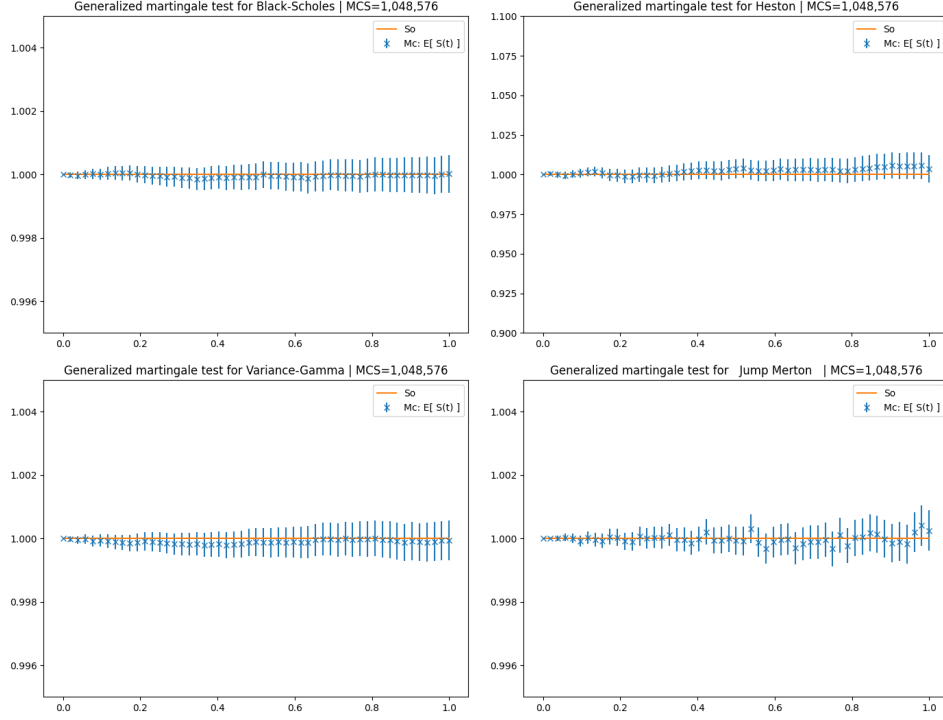


Figure 2: Martingale test for all models

2.1 Asian Options

2.1.1 Black-Scholes

In order to obtain the value for the asian put and call options for each trajectory, we defined the function `asian_BS()`. As the title of the chapter suggests, asian options are path dependent, so the function has to turn the whole martingale trajectory into that of the underlying (*lines 11-15*). After that, the average value of the underlying is computed with the following approximation:

$$\bar{A} \approx \frac{1}{N+1} \sum_{n=0}^N S^j(t_n).$$

```

1 def asian_BS(r, q, k, T, N, J, So, sigma, rand):
2
3     '''
4     Returns the prices of the asian put and call
5     options for each trajectory simulation
6     '''
7     dt = T/N
8     S = BS_trj(rand, N, So, T, J, sigma)
9
10    #from martingale to S
11    D_tT = exp((r-q)*dt)
12    b = 1
13    for n in range(1,N+1):
14        b *= D_tT
15        S[n] *= b
16
17    A_bar = np.sum(S, axis = 0)/(N+1)
18    #A_bar = (.5*So + np.sum(S[1:N-1], axis = 0) + .5*S[N])/N
19
20    #payoff of a call
21    payoff = A_bar - k
22
23    #option value
24    asian_call = exp(-r*T)*np.where(payoff > 0.0, payoff, 0.0)
25    asian_put = exp(-r*T)*np.where(-payoff > 0.0, -payoff, 0.0)
26    return asian_call, asian_put

```

Finally, the Monte Carlo results are computed with the function `mc_asian_BS()`, that follows the same structure of `do_mc()` from CFLib. Moreover, as in exercise 2.4.7 of the lecture notes, we implemented a feature that allows to perform the Monte Carlo simulation with different time frequencies (monthly, weekly and daily).

```

@ So                1.0000
@ Strike            1.2000
@ sigma             0.2000
@ T                 1.0000
@ r                 0.0150
@ q                 0.0000
@ Nt                 52

```

```

@ elapsed time for 16384 iterations: 0.05220 sec.
@ elapsed time for 65536 iterations: 0.11720 sec.
@ elapsed time for 262144 iterations: 0.47802 sec.
@ elapsed time for 1048576 iterations: 2.31750 sec.

```

J	Asian Call +/-	MC-err
16384	0.003863 +/-	1.6399e-04
65536	0.003743 +/-	8.1614e-05
262144	0.003784 +/-	4.0531e-05
1048576	0.003783 +/-	2.0332e-05

```
#----
```

J	Asian Put +/-	MC-err
---	---------------	--------

16384	0.193419 +/-	8.3018e-04
65536	0.193349 +/-	4.1364e-04
262144	0.193569 +/-	2.0779e-04
1048576	0.193340 +/-	1.0374e-04

#----

MC error behavior for different time steps (monthly, weekly, daily)

dt	Asian Call +/-	MC-err
12	0.003588 +/-	1.5541e-04
52	0.003863 +/-	1.6399e-04
365	0.003843 +/-	1.6333e-04

dt	Asian Put +/-	MC-err
12	0.192842 +/-	8.2013e-04
52	0.193419 +/-	8.3018e-04
365	0.193449 +/-	8.3434e-04

The first thing we notice is that the error, for both the call and the put, is cut approximately in half when the number of simulations is increased by four times, so we can affirm that our Monte Carlo simulation is working properly. Moreover the error for the asian put, is way higher than the error of the asian call, and this happens because the put is in the money at the inception of the contract. As a consequence of being in the money, also the price of the put is higher than the one for the call.

2.1.2 Heston

The function we defined to compute the value (and also the Monte Carlo simulation in this case) of the asian options according to the Heston model is `asian_H()`. Inside the function, the volatility trajectories are generated through `QT_cir_evol()`, which is robust to violations of the Feller condition (in our case $F_c = 5.85 > 1$). Moreover, given that generating a volatility trajectory is computationally expensive, the function exploits the tower property of expectation.

```

1 def asian_H(**kwargs):
2
3     '''
4     see the notebook for the definition of the keyword arguments
5     '''
6
7     cir = CIR(kappa = lmdb, sigma = eta, theta = nu_bar, ro = nu_0)
8
9     T0.start()
10    elapsed = 0
11    vol, intVol = QT_cir_evol(rand, cir, L, dt, N, Dt, jv)
12
13    D_tT = exp((r-q)*Dt)
14
15
16    for j in range(jv):
17        T1.start()
18        S = mc_heston( rand, So, vol[:,j], intVol[:,j], cir, rho, Dt, js )
19
20        b = 1
21        for n in range(1,N+1):
22            b *= D_tT

```

```

23         S[n] *= b
24
25         A_bar = np.sum(S, axis = 0)/(N+1)
26
27         # Call payoff
28         payoff = A_bar - k
29
30         # Asian CALL
31         call = exp(q-r*T)*np.where(payoff > 0.0, payoff, 0.0)
32         call = np.mean(call)
33
34         # Asian PUT
35         put = exp(q-r*T)*np.where(-payoff > 0.0, -payoff, 0.0)
36         put = np.mean(put)
37
38         if j == 0:
39             CALL = call / jv
40             Err_CALL = call*call / jv
41
42             PUT = put / jv
43             Err_PUT = put*put / jv
44         else:
45             CALL = CALL + call / jv
46             Err_CALL = Err_CALL + call*call / jv
47
48             PUT = PUT + put / jv
49             Err_PUT = Err_PUT + put*put / jv
50
51
52         elapsed += T1.stop()
53
54         T_end = T0.stop()
55         #print("%6d (%10d) elapsed %8.4f sec. " %(jv, js*jv, T_end))
56
57         Err_CALL = np.maximum(Err_CALL - CALL * CALL, 0.0)
58         Err_CALL = 2*np.sqrt(Err_CALL / jv)
59
60         Err_PUT = np.maximum(Err_PUT - PUT * PUT, 0.0)
61         Err_PUT = 2*np.sqrt(Err_PUT / jv)
62
63         return CALL, Err_CALL, PUT, Err_PUT

```

In order to verify if the tower property is providing benefits in terms of computational time, we designed a test called `asian_tower_test()`, that prints option prices under three scenarios:

1. The number of volatility and underlying trajectories are increased in the same way;
2. The number of underlying trajectories is increased, keeping the number of volatility trajectories fixed;
3. The number of volatility trajectories is increased, keeping the number of underlying trajectories fixed;

JS	JV	JSxJV	Call +/- MC-err	Put +/- MC-err	elapsed
64	64	4096	0.001325 +/- 4.5812e-04	0.217406 +/- 2.9600e-02	0.0842 sec.
256	256	65536	0.001128 +/- 1.3625e-04	0.195506 +/- 1.1016e-02	0.2348 sec.
1024	1024	1048576	0.001137 +/- 5.8315e-05	0.190319 +/- 5.3532e-03	2.0719 sec.
4096	4096	16777216	0.001125 +/- 2.7084e-05	0.191001 +/- 2.6159e-03	26.7667 sec.

#----

JS	JV	JSxJV	Call +/- MC-err	Put +/- MC-err	elapsed
4096	1024	4194304	0.001136 +/- 5.4179e-05	0.190493 +/- 5.3650e-03	6.7263 sec.
16384	1024	16777216	0.001144 +/- 5.4201e-05	0.190434 +/- 5.3594e-03	25.7361 sec.

#----

JS	JV	JSxJV	Call +/- MC-err	Put +/- MC-err	elapsed
1024	4096	4194304	0.001120 +/- 2.8275e-05	0.191096 +/- 2.6193e-03	8.2260 sec.
1024	16384	16777216	0.001116 +/- 1.4162e-05	0.190844 +/- 1.3205e-03	32.7942 sec.

#----

The last two tables clearly show that generating trajectories for the volatility is more expensive than generating trajectories for the underlying. With the same overall number of simulations (volatility + underlying), the computational time of the second scenario 2 is strictly lower than the one of the third scenario 3. Moreover, the time improvement that derives from keeping “JV” low is more evident as the overall number of simulations increases. Finally, from the first table we notice that the error is cut approximately in half when the overall number of simulations (volatility + underlying) increases by sixteen times.

2.1.3 Variance Gamma

The function `asian_VG()`, which computes the value of the asian call and put for all the trajectories, follows the same structure of `asian_BS()`, and therefore we will not display the code. The only difference is in the trajectory generator, which calls the `VG` object and the `vg_evol()` function from `CFLib`. The Monte Carlo results, computed with the function `mc_asian_VG()` are shown below. Finally, we notice that the elapsed time for the highest number of simulations is more than double with respect to Black-Scholes and Heston.

```
@ So          1.0000
@ Strike      1.2000
@ T           1.0000
@ r           0.0150
@ Nt          52
@ eta         0.1494
@ nu          0.0626
@ theta       -0.6635

@ elapsed time for 16384 iterations: 0.13144 sec.
@ elapsed time for 65536 iterations: 0.35455 sec.
@ elapsed time for 262144 iterations: 1.32654 sec.
@ elapsed time for 1048576 iterations: 5.33317 sec.
```

J	Asian Call +/-	MC-err
16384	0.002560 +/-	1.0835e-04
65536	0.002709 +/-	5.7933e-05
262144	0.002668 +/-	2.8594e-05
1048576	0.002668 +/-	1.4328e-05

#----

J	Asian Put +/-	MC-err
16384	0.192642 +/-	9.1539e-04
65536	0.192450 +/-	4.5817e-04
262144	0.192384 +/-	2.2890e-04
1048576	0.192413 +/-	1.1425e-04

#----

Comparing the results with previous models, we see that both the call and the put are slightly underpriced with respect to Black and Scholes, and a bit overpriced with respect to Heston. The errors behave correctly.

2.1.4 Jump Merton

The most interesting result is the price for the asian call, which is equal to 0. This means that the average value of the underlying's trajectory is below the strike for each simulation. With regard to the asian put, instead, the price is the smallest among all the models analysed. Finally we notice that the elapsed time is similar to that of the variance gamma model.

```
@ Initial Asset Price:      1.0000
@ Option Strike Price:      1.2000
@ Maturity [yr]:            1.0000
@ Time-steps:                52
@ BS vol:                    0.2000
@ Jump intensity lambda:     0.0063
@ Jump average size:         0.3175
@ Jump size S.D:             0.0782
@ Risk-free Ir:              0.0150
@ Dividend Yield:            0.0000

@ elapsed time for 16384 iterations: 0.0891 sec.
@ elapsed time for 65536 iterations: 0.2782 sec.
@ elapsed time for 262144 iterations: 1.2251 sec.
@ elapsed time for 1048576 iterations: 5.1924 sec.
```

J	Asian Call +/-	MC-err
16384	0.000000 +/-	0.0000e+00
65536	0.000000 +/-	0.0000e+00
262144	0.000000 +/-	0.0000e+00
1048576	0.000000 +/-	0.0000e+00

#----

J	Asian Put +/-	MC-err
16384	0.189870 +/-	1.4972e-04
65536	0.189606 +/-	7.5156e-05
262144	0.189662 +/-	3.7686e-05
1048576	0.189601 +/-	1.8868e-05

#----

2.2 Barrier Options

The procedure to compute the price for barrier options is equivalent to the one applied for asian options, apart from the payoff computation that takes into account the barriers (see snippet below). For this reason most of the code, including the Monte Carlo part, will not be displayed in order to avoid repetitions.

```
1 min_S = np.min(S, axis = 0)
2 max_S = np.max(S, axis = 0)
```



```

3
4     ST = S[N]
5     payoff = ST - k
6
7     # DOC
8     mask_doc = np.logical_and(payoff > 0, min_S > B1)
9     doc = np.where(mask_doc, payoff, 0)
10    doc *= np.exp((q-r) * T)
11
12    # UOP
13    mask_uop = np.logical_and(-payoff > 0, max_S < Bh)
14    uop = np.where(mask_uop, -payoff, 0)
15    uop *= np.exp((q-r) * T)
16
17    # DB
18    mask_db = np.logical_and(B1 < min_S, max_S < Bh)
19    db = np.where(mask_db, ST, 0)
20    db *= np.exp((q-r) * T)

```

2.2.1 Black-Scholes

After checking that the error is behaving correctly, we can compare the Monte Carlo results with the analytical ones from the lecture notes. Unfortunately such solution is not available for the double barrier option. Both for the down-and-out call and the up-and-in put the results are almost identical to the analytical solutions, and the difference is smaller than the Monte Carlo error.

```

@ So                1.0000
@ Strike            1.2000
@ sigma             0.2000
@ T                 1.0000
@ r                 0.0150
@ q                 0.0000
@ Nt                52
@ Low barrier (B1)  0.40
@ High barrier(Bh)  1.60

```

```

@ elapsed time for 16384 iterations: 0.08377 sec.
@ elapsed time for 65536 iterations: 0.12423 sec.
@ elapsed time for 262144 iterations: 0.51992 sec.
@ elapsed time for 1048576 iterations: 2.34409 sec.

```

```

#-- Analytical DOC = 0.0244

```

J	DOC +/-	MC-err
16384	0.024508 +/-	5.9648e-04
65536	0.024324 +/-	2.9507e-04
262144	0.024515 +/-	1.4723e-04
1048576	0.024368 +/-	7.3476e-05

```

#-- Analytical UOP = 0.2066

```

J	UOP +/-	MC-err
16384	0.207147 +/-	1.2401e-03
65536	0.205908 +/-	6.1555e-04
262144	0.207093 +/-	3.1000e-04
1048576	0.206538 +/-	1.5449e-04

```

#----
      J              DB +/-          MC-err
16384      0.975780 +/-    1.7355e-03
65536      0.977155 +/-    8.6356e-04
262144     0.976567 +/-    4.3343e-04
1048576    0.976806 +/-    2.1629e-04
#----

```

2.2.2 Heston

First, the same observations we made for the asian options, with regard to the benefits of keeping the number of volatility trajectories low, apply also in this case. Second, if we compare the results with the other models, we see that they are quite similar. The only exception is the DOC, which is priced at approximately half of the value obtained for Black and Scholes.

```

      JS      JV      JSxJV      DOC +/- MC-err      UOP +/- MC-err      DB +/- MC-err      elapsed
64      64      4096      0.015920 +/- 4.7737e-03      0.250071 +/- 5.3546e-02      0.917894 +/- 7.0483e-02      0.0822 s.
256     256     65536      0.015591 +/- 2.1658e-03      0.204046 +/- 1.8409e-02      0.981082 +/- 2.2932e-02      0.2459 s.
1024    1024    1048576     0.016085 +/- 1.0964e-03      0.196771 +/- 8.7396e-03      0.990453 +/- 1.0727e-02      2.2165 s.
4096    4096    16777216    0.016132 +/- 5.4906e-04      0.198630 +/- 4.4607e-03      0.988676 +/- 5.4068e-03      28.7460 s.
#----

      JS      JV      JSxJV      DOC +/- MC-err      UOP +/- MC-err      DB +/- MC-err      elapsed
4096    1024    4194304     0.016036 +/- 1.0902e-03      0.197076 +/- 8.7405e-03      0.990136 +/- 1.0725e-02      7.1643 s.
16384   1024    16777216    0.016097 +/- 1.0929e-03      0.196955 +/- 8.7364e-03      0.990240 +/- 1.0721e-02      29.3406 s.
#----

      JS      JV      JSxJV      DOC +/- MC-err      UOP +/- MC-err      DB +/- MC-err      elapsed
1024    4096    4194304     0.016110 +/- 5.4950e-04      0.198764 +/- 4.4649e-03      0.988459 +/- 5.4139e-03      9.9668 s.
1024    16384   16777216    0.016072 +/- 2.7424e-04      0.198058 +/- 2.1986e-03      0.989296 +/- 2.6656e-03      39.9564 s.
#----

```

2.2.3 Variance Gamma

As we can observe from the table below, for all three options the Variance Gamma yields quite similar results to Black and Scholes; the errors are also of the same magnitude. Again, we notice that the elapsed time for the highest number of simulations is more than double with respect to Black-Scholes and Heston.

```

@ So              1.0000
@ Strike          1.2000
@ T              1.0000
@ r              0.0150
@ Nt              52
@ Low barrier (Bl) 0.40
@ High barrier(Bh) 1.60
@ eta             0.1494
@ nu              0.0626
@ theta           -0.6635

@ elapsed time for 16384 iterations: 0.11087 sec.
@ elapsed time for 65536 iterations: 0.41390 sec.
@ elapsed time for 262144 iterations: 1.49482 sec.
@ elapsed time for 1048576 iterations: 5.72311 sec.

```

```

      J              DOC +/-          MC-err
16384      0.024924 +/-    5.3575e-04
65536      0.025429 +/-    2.7385e-04
262144     0.025349 +/-    1.3672e-04
1048576    0.025427 +/-    6.8613e-05
#----

      J              UOP +/-          MC-err
16384      0.208542 +/-    1.3815e-03
65536      0.207698 +/-    6.8550e-04
262144     0.208066 +/-    3.4271e-04

```

1048576	0.207881 +/-	1.7114e-04
#----		
J	DB +/-	MC-err
16384	0.986704 +/-	1.7733e-03
65536	0.987450 +/-	8.8529e-04
262144	0.986607 +/-	4.4330e-04
1048576	0.986813 +/-	2.2168e-04
#----		

2.2.4 Jump Merton

Also in this case, we make the usual comparison with the other models: the DOC is almost equal to the other models, while the double barrier and the UOP display the smallest values among all models. Finally we notice that the elapsed time is similar to that of the variance gamma model.

```
@ Initial Asset Price:      1.0000
@ Option Strike Price:     1.2000
@ Low barrier (Bl):        0.40
@ High barrier(Bh):        1.60
@ Maturity [yr]:           1.0000
@ time-steps:              52
@ Price volatility:         0.2000
@ Jump intensity lambda:    0.0063
@ Jump average size:       0.3175
@ Jump size S.D:           0.0782
@ risk-free Ir:            0.0150
@ Dividend Yield:          0.0000

@ elapsed time for 16384 iterations: 0.1546 sec.
@ elapsed time for 65536 iterations: 0.3112 sec.
@ elapsed time for 262144 iterations: 1.2806 sec.
@ elapsed time for 1048576 iterations: 5.2797 sec.
```

MC	DOC +/-	MC-err
16384	0.024510 +/-	6.0261e-04
65536	0.024379 +/-	2.9902e-04
262144	0.024398 +/-	1.4948e-04
1048576	0.024534 +/-	7.5447e-05
#----		

MC	UOP +/-	MC-err
16384	0.185919 +/-	1.2692e-03
65536	0.185886 +/-	6.3431e-04
262144	0.185392 +/-	3.1637e-04
1048576	0.185607 +/-	1.5835e-04
#----		

MC	DB +/-	MC-err
16384	0.884181 +/-	2.8082e-03
65536	0.881913 +/-	1.4102e-03
262144	0.882574 +/-	7.0435e-04
1048576	0.882629 +/-	3.5208e-04
#----		